

# Housing Prediction

Team 3 : 柏欣妍 王子涵 王一舟 贺佳乐

# 创新点1：文本数据的特征工程

```
# 1. 提取总楼层数
def parse_total_floors(text_data):
    if not isinstance(text_data, str): return 1.0
    match = re.search(r'共(\d+)层', text_data)
    if match:
        floors = float(match.group(1))
        return max(floors, 1.0)
    else:
        return 1.0

# 2. 楼层类别映射为比例值
floor_ratio_mapping = {
    '地下室': 0.0, '底层': 0.0, '低楼层': 0.25,
    '中楼层': 0.5, '高楼层': 0.75, '顶层': 1.0
}

# 3. 提取楼层类别
def extract_floor_category(text_data):
    if not isinstance(text_data, str): return 'Unknown'
    if '地下室' in text_data: return '地下室'
    if '顶层' in text_data: return '顶层'
    if '高楼层' in text_data: return '高楼层'
    if '中楼层' in text_data: return '中楼层'
    if '低楼层' in text_data: return '低楼层'
    if '底层' in text_data: return '底层'
    return 'Unknown'

# 4. 生成三个新特征
X_train['Total_Floors'] = X_train['所在楼层'].apply(parse_total_floors)
X_train['Floor_Ratio'] = X_train['所在楼层'].apply(extract_floor_category).map(floor_ratio_mapping)
X_train['Estimated_Floor'] = X_train['Total_Floors'] * X_train['Floor_Ratio']
```

--- 步骤 3.4.1 (最终修正版): 开始高级量化 '所在楼层' ---  
采纳策略: 将 '低/中/高' 直接映射为比例, 生成交互特征。  
已成功创建 'Total\_Floors' 特征。  
已成功创建 'Floor\_Ratio' 特征。

--- 步骤 3.4.2 (续): 为 'Floor\_Ratio' 填充 NaN ---  
已使用中位数 0.50 填充 'Floor\_Ratio'。  
已成功创建 'Estimated\_Floor' 交互特征!

--- 新的楼层特征 (10个随机样本) ---

|       | 所在楼层       | Total_Floors | Floor_Ratio | Estimated_Floor |
|-------|------------|--------------|-------------|-----------------|
| 59648 | 中楼层 (共34层) | 34.0         | 0.50        | 17.00           |
| 23769 | 低楼层 (共18层) | 18.0         | 0.25        | 4.50            |
| 33600 | 低楼层 (共25层) | 25.0         | 0.25        | 6.25            |
| 41486 | 高楼层 (共18层) | 18.0         | 0.75        | 13.50           |
| 84945 | 高楼层 (共7层)  | 7.0          | 0.75        | 5.25            |
| 3404  | 顶层 (共6层)   | 6.0          | 1.00        | 6.00            |
| 58665 | 中楼层 (共5层)  | 5.0          | 0.50        | 2.50            |
| 39505 | 中楼层 (共32层) | 32.0         | 0.50        | 16.00           |
| 70142 | 低楼层 (共6层)  | 6.0          | 0.25        | 1.50            |
| 6684  | 高楼层 (共26层) | 26.0         | 0.75        | 19.50           |

## 文本解析 + 交互特征

```
def parse_build_year(text_data):
    """解析建筑年代"""
    if not isinstance(text_data, str):
        return np.nan
    # 提取所有4位数字年份
    numbers = re.findall(r'(\d{4})', text_data)
    if len(numbers) == 0:
        return np.nan
    elif len(numbers) == 1:
        return float(numbers[0]) # 单个年份
    else:
        # 年份范围取平均值
        first_year = float(numbers[0])
        last_year = float(numbers[-1])
        return (first_year + last_year) / 2

# 计算房龄
year_extracted = X_train['建筑年代'].apply(parse_build_year)
X_train['House_Age'] = 2025 - year_extracted # 当前年份 - 建筑年份
# 填充缺失值
median_age = X_train['House_Age'].median()
X_train['House_Age'] = X_train['House_Age'].fillna(median_age)
```

--- 处理 '建筑年代' ---  
已创建 'House\_Age', 使用中位数 16.50 填充 NaN

文本特征 (前3行):

|       | House_Age |
|-------|-----------|
| 20486 | 23.5      |
| 53134 | 16.5      |
| 8277  | 36.0      |

## 数值提取 + 时间计算

# 创新点2 - 地理位置降维

## 问题

(1) 板块 (Block) 有 1000+ 唯一值。进行标准独热编码 (OHE) -> 1000+ 新特征列，产生维度灾难！

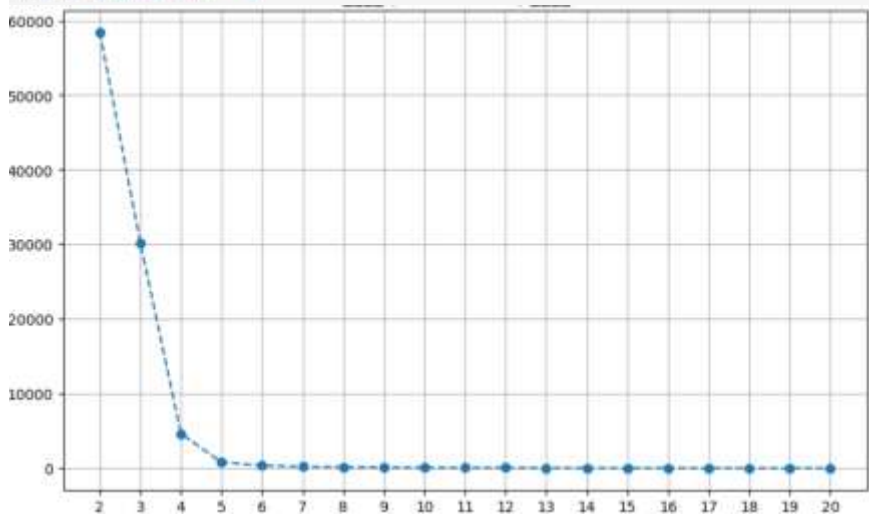
(2) lon/lat (经纬度) -> 线性模型无法理解非线性的空间关系！

```
data_for_elbow = X_train_coords_scaled
print(f"正在使用全部 {len(data_for_elbow)} 个训练样本点进行肘部法则计算...")

wcss = [] # 存储簇内误差平方和 (Inertia)
k_range = range(2, 21) # 尝试 k = 2 到 20

# --- 2. 循环计算不同 k 值的 WCSS ---
for k in k_range:
    print(f"正在计算 k = {k}...")
    # n_init='auto' 是较新版本 sklearn 的推荐设置，它会自动选择 n_init 次数
    # random_state 确保结果可复现
    kmeans_model = KMeans(n_clusters=k, init='k-means++', n_init='auto', max_iter=300, random_state=111)
    kmeans_model.fit(data_for_elbow)
    wcss.append(kmeans_model.inertia_) # inertia_ 就是 WCSS

print("WCSS 计算完毕。")
```



```
# --- 2. 计算每个城市的均值和计数 (仅基于训练集) ---
city_stats = train_data_for_encoding.groupby('城市')[target_col_name].agg(['mean', 'count'])
print(f"从训练集中计算了 {len(city_stats)} 个城市的统计信息。")

# --- 3. 定义平滑参数和全局均值 ---
m = 20 # 平滑因子 (可以调整)
global_mean = global_mean_log_price # 使用之前计算的值

# --- 4. 应用平滑公式计算 TE_City ---
city_stats['TE_City_Smoothed'] = (city_stats['count'] * city_stats['mean'] + m * global_mean) / (city_stats['count'] + m)

# --- 5. 创建编码映射字典 ---
# 我们只需要 'TE_City_Smoothed' 这一列
city_encoding_map = city_stats['TE_City_Smoothed'].to_dict()
print(f"创建了包含 {len(city_encoding_map)} 个城市的编码映射。")
# print(city_encoding_map) # 可以取消注释查看具体的编码值

# --- 6. 将编码映射应用到所有三个数据集 ---
print("正在将 TE_City 应用到 X_train, X_val, X_test...")

# --- 2. 计算每个区域的均值和计数 (仅基于训练集) ---
district_stats = train_data_for_encoding.groupby('区域')[target_col_name].agg(['mean', 'count'])
print(f"从训练集中计算了 {len(district_stats)} 个区域的统计信息。")

# --- 3. 定义平滑参数和全局均值 (保持不变) ---
m = 20 # 平滑因子
# global_mean 应该仍然是 14.2646

# --- 4. 应用平滑公式计算 TE_District ---
district_stats['TE_District_Smoothed'] = (district_stats['count'] * district_stats['mean'] + m * global_mean) / (district_stats['count'] + m)

# --- 5. 创建编码映射字典 ---
district_encoding_map = district_stats['TE_District_Smoothed'].to_dict()
print(f"创建了包含 {len(district_encoding_map)} 个区域的编码映射。")

# --- 6. 将编码映射应用到所有三个数据集 ---
print("正在将 TE_District 应用到 X_train, X_val, X_test...")
```

(1) 目标编码 + 分层平滑：

板块\_ID -> TE\_Block (该板块的平均对数价格)。为防止过拟合，我使用了分层平滑——将样本量小的板块向其所属区域的均值平滑，而不是向全局均值平滑。

(2) K-Means 地理聚类：替换 lon/lat。

肘部法则显示 k=5 是最佳聚类数。我创建了 Geo\_Cluster 特征，将连续的坐标转换为了模型可以理解的、有意义的 5 个地理区域。

# 创新点3：交互项等变换特征的引入

# 1. 区域价值与物业品质的交互

```
if all(col in X_train.columns for col in ['TE_District', 'Property_Fee']):
    X_train_int['TE_District_x_Property_Fee'] = X_train_int['TE_District'] * X_train_int['Property_Fee']
    X_val_int['TE_District_x_Property_Fee'] = X_val_int['TE_District'] * X_val_int['Property_Fee']
    X_test_int['TE_District_x_Property_Fee'] = X_test_int['TE_District'] * X_test_int['Property_Fee']
    print("✓ 添加：区域价值 x 物业品质")
```

协同效应

# 2. 单位面积价值指标

```
if all(col in X_train.columns for col in ['TE_Block_Stratified', '建筑面积']):
    X_train_int['Price_Per_Sqm_Estimate'] = X_train_int['TE_Block_Stratified'] / (X_train_int['建筑面积'] + 1)
    X_val_int['Price_Per_Sqm_Estimate'] = X_val_int['TE_Block_Stratified'] / (X_val_int['建筑面积'] + 1)
    X_test_int['Price_Per_Sqm_Estimate'] = X_test_int['TE_Block_Stratified'] / (X_test_int['建筑面积'] + 1)
    print("✓ 添加：单位面积价值指标")
```

规模效应

模型性能对比：

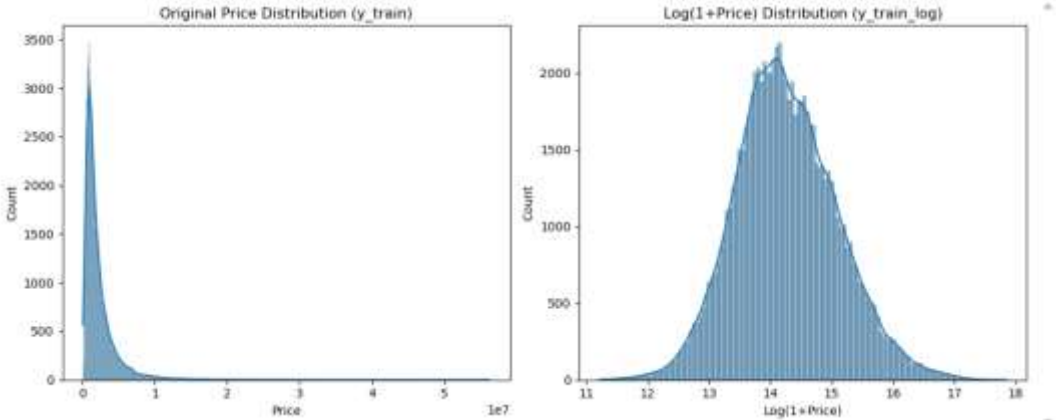
|   | model_name        | val_rmse_原始 | val_rmse_交互项 | RMSE_提升 | val_r2_原始 \ |
|---|-------------------|-------------|--------------|---------|-------------|
| 0 | Linear Regression | 0.2624      | 0.2584       | 0.0040  | 0.8983      |
| 1 | Lasso             | 0.2633      | 0.2592       | 0.0040  | 0.8977      |
| 2 | Ridge             | 0.2624      | 0.2584       | 0.0040  | 0.8983      |
| 3 | Elastic Net       | 0.2625      | 0.2585       | 0.0039  | 0.8983      |

|   | val_r2_交互项 | R2_提升  |
|---|------------|--------|
| 0 | 0.9014     | 0.0031 |
| 1 | 0.9008     | 0.0031 |
| 2 | 0.9014     | 0.0031 |
| 3 | 0.9013     | 0.0030 |

对价格做log变换（右偏分布，变换之后趋于正态）

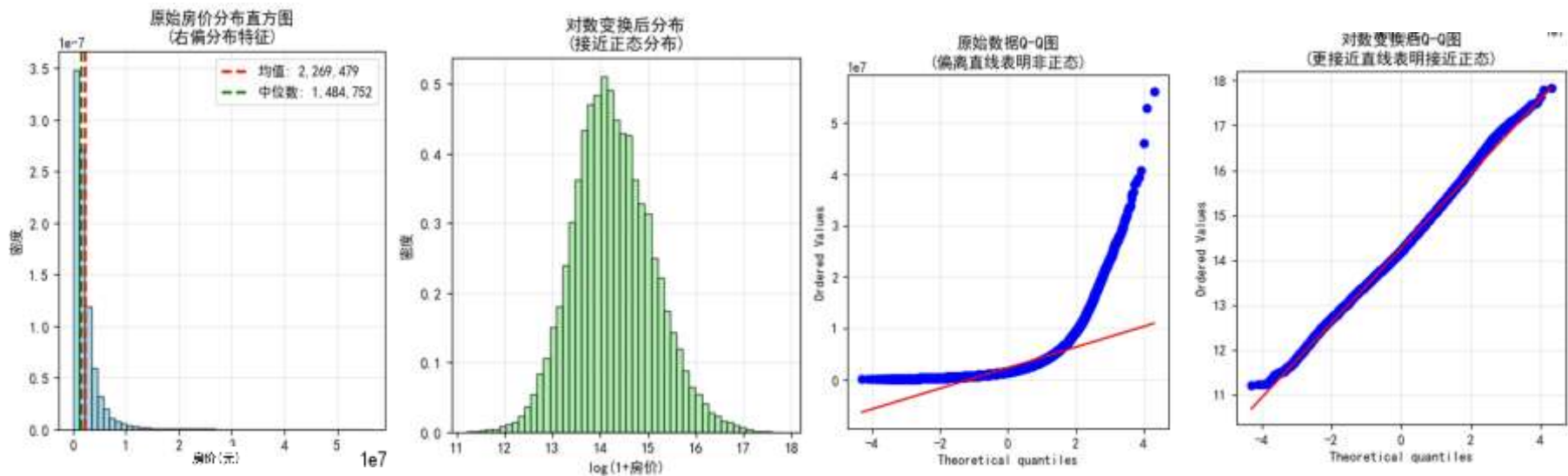
# 目标变量对数转换

```
print("\n--- 步骤7：目标变量对数转换 ---")
y_train_log = np.log1p(y_train)
y_val_log = np.log1p(y_val)
print("目标变量对数转换完成")
```





# 创新点4：对数变换



## 对数变换在OLS模型中的运用效果

用于演示的特征: ['建筑面积', '房龄', '房屋户型\_室', '所在楼层']

| 线性变换组合   | MAE          | RMSE         | R <sup>2</sup> | 系数变异系数 | 截距项           |
|----------|--------------|--------------|----------------|--------|---------------|
| 无变换      | 1,307,126.00 | 2,119,692.90 | 0.3057         | 1.5406 | -1,783,568.86 |
| 目标变量对数变换 | 1,187,487.03 | 2,172,664.11 | 0.3329         | 1.3434 | 12.5926       |

